Day 25 — Capstone Part 1 — Sensor and NVS Integration

Goal

Combine BME280 I2C sensor reading with NVS persistent storage to build the first stage of a battery-powered wireless sensor node.

What you will learn

- How to read temperature and humidity from a BME280 over I2C
- How to mount an NVS filesystem on internal flash
- How to persist a boot counter and last sensor reading across resets
- How to log NVS-backed state at startup for diagnostics

Overview

A production IoT node must survive power cycles gracefully: it should remember how many times it has booted and what the last reading was, even if no central is connected. NVS (Non-Volatile Storage) provides a simple key-value store on internal flash that persists across resets and System OFF sleep.

NVS setup

```
#include <zephyr/fs/nvs.h>
#include <zephyr/storage/flash_map.h>

#define NVS_PARTITION        storage_partition
#define NVS_PARTITION_DEVICE FIXED_PARTITION_DEVICE(NVS_PARTITION)
#define NVS_PARTITION_OFFSET FIXED_PARTITION_OFFSET(NVS_PARTITION)

#define NVS_ID_BOOT_COUNT  1
#define NVS_ID_LAST_TEMP   2
#define NVS_ID_LAST_HUM    3
```

```
static struct nvs_fs fs;

static int nvs_init_fs(void)
{
    struct flash_pages_info info;
    int ret;

    fs.flash_device = NVS_PARTITION_DEVICE;
    fs.offset = NVS_PARTITION_OFFSET;
    ret = flash_get_page_info_by_offs(fs.flash_device, fs.offset,
&info);
    if (ret < 0) {
        return ret;
    }
    fs.sector_size = info.size;
    fs.sector_count = 4U;
    return nvs_mount(&fs);
}
```

Reading the BME280 and persisting data

```
int main(void)
{
    const struct device *bme = DEVICE_DT_GET_ANY(bosch_bme280);
    struct sensor_value temp, hum;
    uint32_t boot_count = 0;
    int32_t last_temp_mdeg = 0, last_hum_m = 0;
    int ret;

    if (!device_is_ready(bme)) {
        LOG_ERR("BME280 not ready");
        return -ENODEV;
    }

    ret = nvs_init_fs();
    if (ret < 0) {
        LOG_ERR("NVS mount failed: %d", ret);
        return ret;
    }

    nvs_read(&fs, NVS_ID_BOOT_COUNT, &boot_count,
sizeof(boot_count));
    nvs_read(&fs, NVS_ID_LAST_TEMP,  &last_temp_mdeg,
sizeof(last_temp_mdeg));
    nvs_read(&fs, NVS_ID_LAST_HUM,   &last_hum_m,
sizeof(last_hum_m));

    LOG_INF("Boot #%u, last T=%d.%03d C, H=%d.%03d %%",
            boot_count,
            last_temp_mdeg / 1000, abs(last_temp_mdeg % 1000),
            last_hum_m / 1000,     abs(last_hum_m % 1000));
```

```
            boot_count++;
            nvs_write(&fs, NVS_ID_BOOT_COUNT, &boot_count,
sizeof(boot_count));

            ret = sensor_sample_fetch(bme);
            if (ret < 0) {
                LOG_ERR("Fetch failed: %d", ret);
                return ret;
            }
            sensor_channel_get(bme, SENSOR_CHAN_AMBIENT_TEMP, &temp);
            sensor_channel_get(bme, SENSOR_CHAN_HUMIDITY,     &hum);

            last_temp_mdeg = temp.val1 * 1000 + temp.val2 / 1000;
            last_hum_m     = hum.val1  * 1000 + hum.val2  / 1000;

            nvs_write(&fs, NVS_ID_LAST_TEMP, &last_temp_mdeg,
sizeof(last_temp_mdeg));
            nvs_write(&fs, NVS_ID_LAST_HUM,  &last_hum_m,
sizeof(last_hum_m));

            LOG_INF("T=%.2f C  H=%.2f %%",
                    sensor_value_to_double(&temp),
                    sensor_value_to_double(&hum));

            return 0;
        }
```

prj.conf

```
CONFIG_I2C=y
CONFIG_SENSOR=y
CONFIG_BME280=y
CONFIG_NVS=y
CONFIG_FLASH=y
CONFIG_FLASH_PAGE_LAYOUT=y
CONFIG_FLASH_MAP=y
CONFIG_LOG=y
CONFIG_MAIN_STACK_SIZE=2048
```

Why this matters

Persistent boot counting and last-reading storage are standard patterns in production firmware. They allow diagnostics (how many times has this node rebooted?), data continuity (what was the last value if connectivity was lost?), and safety (detect abnormal reset loops).

Practice tasks

1. Flash the firmware and open a serial terminal — confirm the boot counter increments on each reset.
2. Power-cycle the board and verify the last temperature and humidity are logged correctly.
3. Corrupt the NVS by calling `nvs_clear(&fs)` and confirm graceful recovery on the next boot.
4. Add a fourth NVS key to store the minimum temperature ever recorded.

Example folder

View the complete source code on GitHub: src/capstone_wireless_sensor_node

Next topic

Day 26 adds a BLE GATT environmental service to broadcast the sensor readings to a phone over Bluetooth.